

```

register int i,j;
pas[0][0]=1;
pas[1][0]=pas[1][1]=1;
for(i=2;i<=n;i++)
{
    pas[i][0]=1;
    for(j=1;j<i;j++)
    {
        pas[i][j]= pas[i-1][j-1]+pas[i-1][j];
    }
    pas[i][j]=1;
}
}

main(void)
{
    pascals(10);
    int n,m;
    while(scanf("%d%d",&n,&m)!=EOF)
    {
        printf("%lu",pas[n][m]);
    }
    return 0;
}

```

Combination with repeated objects :

If in a word of s length character, a character is repeated l times, then the number of arrangement is:

$$\frac{s!}{l!}$$

If there is more then one repeated character the we can write,

$$\frac{s!}{l_1! * l_2! * \dots * l_n!}$$

where,

l_1 , is the repeation times of first repeated character.

l_2 , is for second repeated character

and, so on...

but remember calculating both $n!$ and $l_1! * l_2! * l_3! * \dots * l_n!$ may occurs overflow. So I use

$$\frac{s!}{1! * 2! * \dots * n!}$$

$$= \frac{1 * 2 * 3 * \dots * s}{1 * 2 * \dots * 1 * 1 * 2 * 3 * \dots * 2 * \dots * 1 * 2 * \dots * n}$$

$$= \frac{1}{1} * \frac{2}{2} * \dots * \frac{m}{1} * \frac{m+1}{1} * \frac{m+2}{2} * \dots * \frac{m2}{12} * \dots * \frac{s}{n}$$

The code of this type of combination is as follows:

```
#include<stdio.h>
#include<math.h>
#include<string.h>

#define MAX 30

/* ***** */
/* A sample function that calculate how many ways that you can */
/* rearrange a word with its letter */
/* ***** */
double test(char *str)
{
    int de[MAX]={0};
    int ss[300] = {0};
    int l = strlen(str);
    int i,j=0;
    double c=1,d=1;
    for(i=0;i<l;i++)
    {
        ss[str[i]]++;
        if(ss[str[i]] > 1)
            de[j++] = ss[str[i]];

    }
    c = 1;
    for(i=2;i<=l;i++)
    {
        c*=i;

        if(j>0)
            d*= de[--j];
        if((d!=1) && !(fmod(c,d)))
        {
            c /= d;
            d=1;
        }
    }
    return c;
}

/* A sample main function */
```

```

main(void)
{
    char word[MAX];
    int n;
    int j=0;
    scanf("%d", &n);
    for(;n>0;n--)
    {
        scanf("%s", word);
        printf("Data set %d: %.0f", ++j, test(word));
        putchar('\n');
    }
    return 0;
}

```

longest common subsequence (LCS)

Given two sequence X and Y, we say that a sequence z is a **common subsequence** of C and Y if Z is a subsequence of both X and Y.

longest common subsequence (LCS) is just the longest "common subsequence" of two sequences.

LCS LENGTH(X,Y)

Input two sequence $X\langle x_1, x_2, x_3, \dots, x_m \rangle$ and $Y\langle y_1, y_2, y_3, \dots, y_n \rangle$. It stores the $C[i, j]$ values in the table $C[0\dots m, 0\dots n]$ whose entries are computed in row major order. It also maintain the table $b[1\dots m, 1\dots n]$ to simplify construction of an optimal solution.

```

1. m <- length[X]
2. n <- length[Y]
3. for i <- 1 to m
4.     do c[i,0] <- 0
5. for j <- 0 to n
6.     do c[0,j] <- 0
7. for i <- 1 to m
8.     for j <- 1 to n
9.         do if  $x_i = y_j$ 
10.            then  $c[i, j] <- c[i - 1, j - 1] + 1$ 
11.                 $b[i, j] <- 1$ 
12.            else if  $c[i - 1, j] \geq c[i, j - 1]$ 
13.                then  $c[i, j] <- c[i - 1, j]$ 
14.                     $b[i, j] <- 2$ 
15.            else  $c[i, j] <- c[i, j - 1]$ 
16. return c and b

```

PRINT_LCS(b,X,i,j)

```

1. if i = 0 or j = 0
2.     then return
3. if b[i,j] = 1
4.     then PRINT_LCS(b,X,i-1,j-1)
5. else if b[i,j] = 2
6.     then PRINT_LCS(b,X,i-1,j)
7. else PRINT_LCS(b,X,i,j-1)

```

Code for LCS

```

#include<stdio.h>
#include<string.h>

#define MAX 100 // size of each sequence

char str1[MAX],str2[MAX]; // the sequences

/* ***** */
/* This function print the LCS to the screen */
/* Input : A table generated by the LCS_LNT */
/* and the length of the sequences */
/* Output: None */
/* ***** */

void p_lcs(int b[MAX][MAX],int i,int j)
{
    if ( (i == 0) || (j == 0) ) return ;
    if( b[i][j] == 1 )
    { p_lcs(b,i-1,j-1);
      printf("%3c",str1[i-1]);
    }
    else if( b[i][j] == 2) p_lcs(b,i-1,j);
    else p_lcs(b,i,j-1);
}

/* ***** */
/* This function calculate the LCS length */
/* Input : Two Sequence and an bool I. If */
/* I is FALSE(0) then the function */
/* do not print the LCS and if */
/* TRUE(1) then print using the */
/* above p_lcs function */
/* Output: None */
/* ***** */

void LCS_LNT(bool I)
{
    int c[MAX][MAX]={0},b[MAX][MAX]={0},l1,l2;
    l1 = strlen(str1)+1;
    l2 = strlen(str2)+1;

```

```

    register int i,j;
    for(i=1;i<11;i++)
    {   for(j=1;j<12;j++)
        {   if( str1[i-1] == str2[j-1] )
            {   c[i][j] = c[i-1][j-1] + 1;
                b[i][j] = 1;
            }
            else if(c[i-1][j] >= c[i][j-1])
            {   c[i][j] = c[i-1][j];
                b[i][j] = 2;
            }
            else   c[i][j] = c[i][j-1];
        }
    }
    printf("%d\n",c[11-1][12-1]);
    if(I) p_lcs(b,11-1,12-1);
}

/* a sample main function */
main(void)
{
    while(1)
    {
        if(!(gets(str1)))        return 0;
        if(!(gets(str2)))        return 0;
        LCS_LNT(1);
    }
}

```

Another code for finding LCS

```

#include<stdio.h>
#include<string.h>
#define MAX 105

char str1[MAX][50],str2[MAX][50],lcs[MAX][50];
int lcswl;

/* ***** */
/*   This function print the LCS to the screen   */
/*   Input  : A table generated by the LCS_LNT   */
/*             and the length of the sequences   */
/*   Output: None                               */
/* ***** */
void p_lcs(int b[MAX][MAX],int i,int j)
{
    if ( (i == 0) || (j == 0) ) return ;
    if( b[i][j] == 1 )
    {
        p_lcs(b,i-1,j-1);
        strcpy(lcs[lcswl++],str1[i-1]);
    }
    else if( b[i][j] == 2)                p_lcs(b,i-1,j);
}

```

```

    else
        p_lcs(b,i,j-1);
}

/* ***** */
/* This function calculate the LCS length */
/* Input : Tow Sequence and an bool I. If */
/* I is FALSE(0) then the function */
/* do not print the LCS and if */
/* TRUE(1) then print using the */
/* above p_lcs function */
/* Output: None */
/* ***** */
void LCS_LNT(int l1, int l2,bool I)
{
    int c[MAX][MAX]={0},b[MAX][MAX]={0};
    register int i,j;
    for(i=1;i<l1;i++)
    {
        for(j=1;j<l2;j++)
        {
            if( !(strcmp(str1[i-1],str2[j-1])) )
            {
                c[i][j] = c[i-1][j-1] + 1;
                b[i][j] = 1;
            }
            else if(c[i-1][j] >= c[i][j-1])
            {
                c[i][j] = c[i-1][j];
                b[i][j] = 2;
            }
            else
                c[i][j] = c[i][j-1];
        }
    }
    if(I)
    {
        lcswl = 0;
        p_lcs(b,l1-1,l2-1);
        j = c[l1-1][l2-1];
        printf("%s",lcs[0]);
        for(i = 1; i < j ;i++)
            printf(" %s",lcs[i]);
        putchar('\n');
    }
}

/* Sample main function */
main(void)
{
    char word[50];
    int i=0,j=0,l1,l2,ln;
    while(scanf("%s",word) != EOF)
    {
        ln = strlen(word);
        if(ln==1)
            if(word[0] == '#')
            {
                if(i==0)
                {
                    i = 1;

```

```

        l1 = j;
        j = 0;
        continue;
    }
    else
    {
        l2 = j;
        j = i = 0;
        test(l1+1,l2+1,1);
        continue;
    }
}
if(i==0) strcpy(str1[j++],word);
else strcpy(str2[j++],word);
}
return 0;
}

```

Matrix Chain Multiplication (MCM)

Background

If two matrix A and B whose dimension is (m,n) and (n,p) respectively then the multiplication of A and B needs $m*n*p$ scalar multiplication.

Suppose you are given a sequence of n matrix $A_1, A_2, \dots, A_n$. Matrix A_i has dimension (P_{i-1}, P_i) . Your task is to parenthesize the product $A_1, A_2, \dots, A_n$ such a way that minimize the number of scalar product.

As matrix multiplication is associative so all the parenthesizations yield the same product.

MATRIX CHAIN ORDER(P)

The input is a sequence $P = \langle P_0, P_1, P_2, \dots, P_n \rangle$ where $\text{length}[P] = n+1$. The procedure uses an auxlary table $m[1 \dots n, 1 \dots n]$ for storing the $m[i, j]$ costs and auxlary table $s[1 \dots n, 1 \dots n]$ that records which index of k achive the optimal cost in computing $m[i, j]$.

```

1. n <- length[P]-1
2. for i <- 1 to n
3.     do m[i,j] <- 0
4. for l <- 2 to n
5.     do for i <- 1 to n - l + 1
6.         do j <- i + l - 1

```

```

7.          m[i,j] <- INF      // INF is infinity
8.          for k <- i to j -1
9.            do q <- m[i,k]+m[k+1,j]+ Pi-1PkPj
10.           if q < m[i,j]
11.             then m[i,j] <- q
12.              s[i,j] <- k
13. return m,s

```

PRINT(s,i,j)

The following recursive procedure prints an optimal parentthesization of $(A_i, A_{i+1}, \dots, A_j)$ given the s table computed by

MATRIX CHAIN ORDER and indices i and j. The initial call PRINT(s,1,n) prints optimal parentthesization of $(A_1, A_2, \dots, A_n)$.

```

1.  if i == j
2.    then print "A"i
3.    else print "("
4.         PRINT(s,i,s[i,j])
5.    PRINT(s,s[i,j]+1,j)
6.    print ")"

```

Code :

```

#include<stdio.h>
#define MAX 15          /* number of matrix */
#define INF 4294967295  /* a maximum number of multiplication */

int num;
/* ***** */
/*      Print out the sequence          */
/*      Input : A two dimentional array(created          */
/*              by the matrix chan order function)        */
/*              with there starting point                */
/*      Return : None                                */
/* ***** */
void print(unsigned long s[][MAX],int i,int j)
{
    if(i==j)
        printf("A%d",num++);
    else
    {
        printf("(");
        print(s,i,s[i][j]);
        printf(" x ");
        print(s,s[i][j]+1,j);
        printf(")");
    }
}

```

```

/* ***** */
/* Find out the order */
/* Input : A sequence of dimension and the */
/*          number of matrix */
/* Return : none */
/* ***** */

void matrix_chan_order(int *p,int n)
{
    unsigned long m[MAX][MAX] = {0};
    unsigned long s[MAX][MAX] = {0};
    unsigned int q;
    int l,j,i,k;
    for(l = 2; l <= n ;l++)
    {
        for(i = 1 ; i <= n - l +1 ; i++)
        {
            j = i + l -1;
            m[i][j] = INF;
            for(k=i ; k < j ; k++) {
                q = m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j];
                if(q < m[i][j])
                {
                    m[i][j] = q;
                    s[i][j] = k;
                }
            }
        }
    }
    num =1;
    print(s,1,n);
}

/* A sample main function */

main(void)
{
    int n;
    int p[MAX]={0};
    int i;

    while(scanf("%d",&n),n)
    {
        for(i=1;i<=n;i++)
            scanf("%d %d",&p[i-1],&p[i]);
        matrix_chan_order(p,n);
        putchar('\n');

    }
    return 0;
}

```

Big Number Addition

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<math.h>
#define MAX 1000
void reverse(char *from, char *to ){
    int len=strlen(from);
    int l;
    for(l=0;l<len;l++){
        to[l]=from[len-l-1];
    }
    to[len]='\0';
}
void call_sum(char *first, char *sec, char *result){
    char F[MAX], S[MAX], Res[MAX];
    int f,s,sum,extra,now;
    f=strlen(first);
    s=strlen(sec);
    reverse(first,F);
    reverse(sec,S);
    for(now=0,extra=0;(now<f && now<s);now++){
        sum=(F[now]-'0') + (S[now]-'0') + extra;
        Res[now]=sum%10 +'0';
        extra= sum/10;
    }
    for(;now<f;now++){
        sum=F[now] + extra-'0';
        Res[now]=sum%10 +'0';
        extra=sum/10;
    }
    for(;now<s;now++){
        sum=S[now] + extra-'0';
        Res[now]=sum%10 +'0';
        extra=sum/10;
    }
    if(extra!=0) Res[now++]=extra+'0';
    Res[now]='\0';
    if(strlen(Res)==0) strcpy(Res,"0");
    reverse(Res,result);}

int main(){
    char fir[MAX],sec[MAX],res[MAX];
    while(scanf("%s%s",&fir,&sec)==2){
        call_sum(fir,sec,res);
        int len=strlen(res);
        for(int i=0;i<len;i++) printf("%c",res[i]);
        printf("\n");
    }
    return 0;
}
```

Big Number Subtraction

```

/*****      Big Number Subtraction      *****/
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<math.h>
#define MAX 1000
/*****
void reverse(char *from, char *to ){
    int len=strlen(from);
    int l;
    for(l=0;l<len;l++){
        to[l]=from[len-l-1];
    }
    to[len]='\0';
}
int call_minus(char *large, char *small, char *result){
    char L[MAX], S[MAX];
    int l,s,now,hold,diff;
    l=strlen(large);
    s=strlen(small);
    bool sign = 0;
    if(l<s){
        strcpy(result,large);
        strcpy(large,small);
        strcpy(small,result);
        now=l; l=s; s=now;
        sign = 1;
    }//return 0;
    if(l==s){
        if(strcmp(large, small)<0){
            strcpy(result,large);
            strcpy(large,small);
            strcpy(small,result);
            now=l; l=s; s=now;
            sign =1;
        }//return 0;
    }
    reverse(large,L);
    reverse(small,S);
    for(;s<l;s++)
        S[s]='0';
    S[s]='\0';
    for(now=0,hold=0;now<l;now++){
        diff=L[now]-(S[now]+hold);
        if(diff<0){
            hold=1;
            result[now]=10+diff+'0';
        }
        else{
            result[now]=diff+'0';
            hold=0;
        }
    }
}

```

```

    }
    for(now=l-1;now>0;now--){
        if(result[now]!='0')
            break;
    }
    result[now+1]='\0';
    reverse(result,L);
    strcpy(result,L);
    //return 1;
    return sign;
}
int main(){
    char fir[MAX],sec[MAX],res[MAX];
    while(scanf("%s%s",&fir,&sec)==2){
        if(call_minus(fir,sec,res)==1)
            printf("-");
        int len = strlen(res);
        for(int i=0;i<len;i++)
            printf("%c",res[i]);
        printf("\n");

    }
    return 0;
}

```

Big Number Multiplication

```

/*****      Big Number Multiplication      *****/
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<math.h>
#define MAX 1000
/*****
void reverse(char *from, char *to ){
    int len=strlen(from);
    int l;
    for(l=0;l<len;l++)
        to[l]=from[len-l-1];
    to[len]='\0';
}
*****/
/*****
void call_mult(char *first,char *sec,char *result){
    char F[MAX],S[MAX],temp[MAX];
    int f_len,s_len,f,s,r,t_len,hold,res;
    f_len=strlen(first);
    s_len=strlen(sec);
    reverse(first,F);
    reverse(sec,S);
    t_len=f_len+s_len;
    r=-1;

```

```

    for(f=0;f<=t_len;f++)
        temp[f]='0';
    temp[f]='\0';
    for(s=0;s<s_len;s++){
        hold=0;
        for(f=0;f<f_len;f++){
            res=(F[f]-'0')*(S[s]-'0') + hold+(temp[f+s]-'0');
            temp[f+s]=res%10+'0';
            hold=res/10;
            if(f+s>r) r=f+s;
        }
        while(hold!=0){
            res=hold+temp[f+s]-'0';
            hold=res/10;
            temp[f+s]=res%10+'0';
            if(r<f+s) r=f+s;
        }
        f++;
    }
    for(;r>0 && temp[r]=='0';r--);
    temp[r+1]='\0';
    reverse(temp,result);
}
/*****
int main(){
    char fir[MAX],sec[MAX],res[MAX];
    while(scanf("%s%s",&fir,&sec)==2){
        call_mult(fir,sec,res);
        int len=strlen(res);
        for(int i=0;i<len;i++) printf("%c",res[i]);
        printf("\n");
    }
    return 0;
}

```

Big Number Division and Remainder

```

/*****          Big Number division          *****/
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<math.h>
#define MAX 1000
/*****
int call_div(char *number,long div,char *result){
    int len=strlen(number);
    int now;
    long extra;
    char Res[MAX];
    for(now=0,extra=0;now<len;now++){
        extra=extra*10 + (number[now]-'0');

```

```

        Res[now]=extra / div +'0';
        extra%=div;
    }
    Res[now]='\0';
    for(now=0;Res[now]!='0';now++);
    strcpy(result, &Res[now]);
    if(strlen(result)==0)
        strcpy(result, "0");
    return extra;
}
/*****
int main(){
    char fir[MAX],res[MAX];
    long sec,remainder;
    while(scanf("%s%ld",&fir,&sec)==2){
        if(sec==0) printf("Divide by 0 error\n");
        else{
            remainder=call_div(fir,sec,res);
            int len=strlen(res);
            for(int i=0;i<len;i++) printf("%c",res[i]);
            printf("\t%ld",remainder);
            printf("\n");
        }
    }
    return 0;
}

```

Big Number Square Root

```

/*****      Big Number Sqrt      *****/
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<math.h>
#define MAX 1000
/*****
int call_minus(char *large, char *small, char *result){
    char L[MAX], S[MAX];
    int l,s,now,hold,diff;
    l=strlen(large);
    s=strlen(small);
    if(l<s)
        return 0;
    if(l==s){
        if(strcmp(large, small)<0)
            return 0;
    }
    reverse(large,L);
    reverse(small,S);
    for(;s<l;s++)
        S[s]='0';

```

```

    S[s]='\0';
    for(now=0,hold=0;now<1;now++){
        diff=L[now]-(S[now]+hold);
        if(diff<0){
            hold=1;
            result[now]=10+diff+'0';
        }
        else{
            result[now]=diff+'0';
            hold=0;
        }
    }
    for(now=1-1;now>0;now--){
        if(result[now]!='0')
            break;
    }
    result[now+1]='\0';
    reverse(result,L);
    strcpy(result,L);
    return 1;
}
/*****
void call_sqrt(char *number,char *result,char *extra){
    int num,start,e,mul,l,r=0,len;
    char left[MAX],after[MAX];
    char who[5],temp[MAX],two[5];
    len=strlen(number);
    if(len%2==0){
        num=10*(number[0]-'0') + number[1]-'0';
        start=2;
    }
    else{
        num=number[0]-'0';
        start=1;
    }
    mul=(int) sqrt(num);
    result[0]=mul+'0';
    result[1]='\0';
    if(num-mul*mul ==0)
        extra[0]='\0';
    else
        sprintf(extra,"%d",num-mul*mul);
    for(;start<len;start+=2){
        e=strlen(extra);
        extra[e]=number[start];
        extra[e+1]=number[start+1];
        extra[e+2]='\0';
        two[0]='2';
        two[1]='\0';
        call_mult(result,two,left);
        l=strlen(left);
        for(mul=9;mul>=0;mul--){
            who[0]=mul+'0';
            who[1]='\0';

```

```

        strcat(left,who);
        call_mult(left,who,after);
        if(call_minus(extra,after,temp)==1){
            result[++r]=mul+'0';
            result[r+1]='\0';
            strcpy(extra,temp);
            break;
        }
    else
        left[l]='\0';

    }
    result[++r]='\0';
}
/*****
int main(){
    char fir[MAX],ex[MAX],res[MAX];
    while(scanf("%s",&fir)==1){
        call_sqrt(fir,res,ex);
        int len=strlen(res);
        for(int i=0;i<len;i++) printf("%c",res[i]);
        printf("\n");
    }
    return 0;
}

```

Fibonacci Numbers

```

/* This run O(log n) time//
#include<stdio.h>
long conquer_fibonacci(long n){
    long i,h,j,k,t;
    i=h=1;
    j=k=0;
    while(n>0){
        if(n%2==1){
            t=j*h;
            j=i*h + j*k +t;
            i=i*k + t;
        }
        t=h*h;
        h=2*k*h + t;
        k=k*k + t;
        n=(long) n/2;}
    return j;}
int main(){
    long n,res;
    while(scanf("%ld",&n)==1){
        res=conquer_fibonacci(n);
        printf("%ld\n",res);}
    return 0;
}

```

Fibonacci Number by Big Integer

```
#include<iostream.h>
#include<string.h>
int main()
{
    char *fibonacci[5001]={0};
    fibonacci[0]="0";
    fibonacci[1]="1";
    int l1=strlen(fibonacci[0]);
    int l2=strlen(fibonacci[1]);
    int l;
    for(long i=2;i<=5000;i++)
    {
        char str[10000];
        if(l1>=l2)l=l1;
        else    l=l2;
        int ca=0;
        long j,k,m,p;
        for(j=l1-1,k=l2-1,m=0,p=0;p<l;j--,k--,m++,p++)
        {
            int s1;
            if(j<0) fibonacci[i-2][j]='0';
            s1=fibonacci[i-2][j]-48;
            int s2;
            if(k<0) fibonacci[i-1][k]='0';
            s2=fibonacci[i-1][k]-48;
            int ans=0;
            ans+=s1+s2+ca;
            if(ans>9)
            {
                str[m]=(ans-10)+48;
                ca=1;
            }
            else
            {
                str[m]=ans+48;
                ca=0;
            }
        }
        if(ca>0){str[m]=ca+48; m++;}
        str[m]='\0';
        fibonacci[i]=new char[m+1];
        long y=0;
        for(long x=m-1;x>=0;x--,y++) fibonacci[i][y]=str[x];
        fibonacci[i][y]='\0';
        l1=strlen(fibonacci[i-1]);
        l2=strlen(fibonacci[i]);
    }
}
```

```

    int n;

    while(cin>>n)
    {
        cout<<"The Fibonacci number for "<<n<<" is "<<fibonacci[n]<<"\n";
    }
    return 0;
}

```

BFS/DFS (Maze Problem)

Input :

```

8 8
#.#.#.#.
.....
#.#.....
#...#...
..#..##.
#..##...
...#...#
#.s#d.#.

```

```

#include<stdio.h>
#include<conio.h>
#include<values.h>

#define N 100
#define MAX MAXINT

int mat[N][N], Q[N][3], cost[N][N], front = -1, rear = -1;
int m, n, sc, sr, p, nr, nc, r, c, leaf, i, j, res[10][10];
int R[4] = {0, -1, 0, 1};
int C[4] = {-1, 0, 1, 0};

void nQ(int r, int c, int p)
{
    Q[++rear][0] = r, Q[rear][1] = c, Q[rear][2] = p;
}

void dQ(int *r, int *c, int *p)
{
    *r = Q[++front][0], *c = Q[front][1], *p = front;
}

void bfs(int sr, int sc, int p)
{
    nQ(sr, sc, p);
    do{
        dQ(&r, &c, &p);
        for(int i=0; i<4; i++)
        {

```

```
        nr = r + R[i], nc = c + C[i];
        if(mat[nr][nc]==1)
        {
            if(cost[nr][nc]>cost[r][c]+1)
                cost[nr][nc]=cost[r][c]+1, nQ(nr, nc, p);
        }
    }
    } while (rear!=front);
    leaf = p;
}
void show()
{
    for(int i=0; i<=m; i++)
    {
        for(int j=0; j<=n; j++)
        {
            if(res[i][j])
                printf("X");
            else
                printf(" ");
        }
        printf("\n");
    }
}
void dfs(int leaf)
{
    if(Q[leaf][2]==-1)
    {
        res[Q[leaf][0]][Q[leaf][1]] = 1;
        return;
    }
    dfs(Q[leaf][2]);
    res[Q[leaf][0]][Q[leaf][1]] = 1;
}
void main()
{
    clrscr();
    char ch;
    freopen("maze.txt", "r", stdin);
    scanf("%d%d", &m, &n);
    getchar();
    for(i=0; i<m; i++)
    {
        for(j=0; j<n; j++)
        {
            cost[i][j] = MAX;
            scanf("%c", &ch);
            if(ch=='#')
                mat[i][j] = 0;
            else if(ch=='.')
                mat[i][j] = 1;
            else if(ch=='s')
```

```

        mat[i][j] = 2, sc = j, sr = i;
    else
        mat[i][j] = 3, res[i][j] = 1;
    }
    getchar();
}
bfs(sr, sc, -1);
dfs(leaf);    show();
}

```

MinSum (DFS/QUEUE)

Input :

```

5 6
3 4 1 2 8 6
6 1 8 2 7 4
5 9 3 9 9 5
8 4 1 3 2 6
3 7 2 1 2 3

```

Code

```

#include<stdio.h>
#include<conio.h>
#include<values.h>

#define N 100
#define MAX MAXINT

int mat[N][N], M[N][N], Q[N][4], front = -1, rear = -1;
int m, n, nc, nr, p, s, finalSum=MAX, leaf, r, c, i;

void init()
{
    for(int i=0; i<N; i++)
        for(int j=0; j<N; j++)
            M[i][j] = MAX, mat[i][j] = 0;
}

void nQ(int r, int c, int p, int s)
{
    Q[++rear][0] = r;    Q[rear][1] = c;    Q[rear][2] = p;
    Q[rear][3] = s;
}

void dQ(int *r, int *c, int *p, int *s)
{
    *r = Q[++front][0];
    *c = Q[front][1];
    *p = front;
    *s = Q[front][3];
}

```

```
void bfs()
{
    for(r=0, c=0; r<m; r++)
        nQ(r, c, -1, mat[r][c]);
    do
    {
        dQ(&r, &c, &p, &s);
        if(c<n-1)
            for(i=-1; i<2; i++)
            {
                nr=(m+r+i)%m, nc=c+1;
                if(M[nr][nc] > s+mat[nr][nc])
                    nQ(nr, nc, p, s+mat[nr][nc]), M[nr][nc] = s+mat[nr][nc];
            }
        else if(s<finalSum)
            finalSum = s, leaf = p;
    } while(rear!=front);
}

void dfs(int leaf)
{
    if(Q[leaf][2]==-1)
    {
        printf(" <%d, %d>", Q[leaf][0]+1, Q[leaf][1]+1);
        return;
    }
    dfs(Q[leaf][2]);
    printf(" <%d, %d>", Q[leaf][0]+1, Q[leaf][1]+1);
}

void main()
{
    clrscr();
    int i, j, t;
    init();
    freopen("in.txt", "r", stdin);
    scanf("%d%d", &m, &n);
    for(i=0; i<m; i++)
        for(j=0; j<n; j++)
        {
            scanf("%d", &t);
            mat[i][j] = t;
        }
    bfs();

    printf("Final sum: %d\nPath:", finalSum);

    dfs(leaf);
}
```

Floyed Warshal**Input**

```
5 7
1 2 4
1 3 1
1 5 6
2 5 3
2 4 1
3 2 1
4 5 1
0 0
```

Code :

```
#include<stdio.h>
#include<values.h>

#define N 100
#define INF MAXINT

int mat[N][N], path[N][N], n, e;

void initMat()
{
    for(int i=1; i<=n; i++)
        for(int j=1; j<=n; j++)
            mat[i][j] = INF;
}

void initPath()
{
    for(int i=1; i<=n; i++)
        for(int j=1; j<=n; j++)
            if(mat[i][j]!=INF)
                path[i][j] = j;
            else
                path[i][j] = 0;
}

void floyd_warshall()
{
    for(int k=1; k<=n; k++)
        for(int i=1; i<=n; i++)
            for(int j=1; j<=n; j++)
                if(mat[i][k]!=INF && mat[k][j]!=INF)
                    if(mat[i][k]+mat[k][j] < mat[i][j])
                        mat[i][j] = mat[i][k] + mat[k][j],
                        path[i][j] = path[i][k];
}
```

```
}  
  
void showPath(int i, int j)  
{  
    if(i==j)  
    {  
        printf("->%d", i);  
        return;  
    }  
    printf("->%d", i);  
    showPath(path[i][j], j);  
}  
  
void main()  
{  
  
    while(scanf("%d%d", &n, &e) && n && e)  
  
    {  
  
        initMat();  
        for(int i, j, c, k=0; k<e; k++)  
        {  
            scanf("%d%d%d", &i, &j, &c);  
            mat[i][j] = c;  
        }  
        initPath();  
        floyd_warshall();  
        for(i=1; i<=n; i++)  
        {  
            for(j=1; j<=n; j++)  
                if(path[i][j])  
                {  
                    printf("%d", i);  
                    showPath(path[i][j], j);  
                    printf("\n");  
                }  
            printf("\n");  
        }  
    }  
}
```

Graph Coloring

```
#include<stdio.h>  
int a[20][20],x[20],n,m;  
void next(int k)  
{  
    int j;  
    while(1)
```

```

    {
        x[k]=(x[k]+1)%(m+1);
        if(x[k]==0)
            return;
        for(j=1;j<=n;j++)
            if(a[k][j]!=0&& x[k]==x[j])
                break;
        if(j==n+1)
            return;
    }
}
void mcolor(int k)
{
    int j;
    while(1)
    {
        next(k);
        if(x[k]==0)
            return;
        if(k==n)
        {
            printf("      ");
            for(j=1;j<=n;j++)
                printf("%2d",x[j]);
        }
        else
            mcolor(k+1);
    }
}
void main()
{
    int i,u,v;
    printf("\n\n Enter how many colors : ");
    scanf("%d",&m);
    printf("\n\n Enter how many nodes(0<n<20) :");
    scanf("%d",&n);
    printf("\n\n Enter your edges(ex- u sp v) (press 'e' for end) : \n");
    for(i=1;i<=(n*n)/2;i++)
    {
        if(getchar()=='e')
            break;
        scanf("%d%d",&u,&v);
        a[u][v]=a[v][u]=1;
    }mcolor(1); printf("\n\n");}

```

Cycle Detection (Hamiltonian)

```

#include<stdio.h>
int a[20][20],x[20],n;
void next(int k)
{
    int j;

```

```
while(1)
{
    x[k]=(x[k]+1)%(n+1);
    if(x[k]==0)
        return;
    if((a[x[k-1]][x[k]])!=0)
    {
        for(j=1;j<=k-1;j++)
            if(x[k]==x[j])
                break;
        if(j==k)
            if((k<n)|| (k==n&&a[x[n]][x[1]]!=0))
                return;
    }
}
}
void hamilt(int k)
{
    int j;
    while(1)
    {
        next(k);
        if(x[k]==0)
            return;
        if(k==n)
        {
            printf("      ");
            for(j=1;j<=n;j++)
                printf("%2d",x[j]);
        }
        else
            hamilt(k+1);
    }
}
void main()
{
    int i,u,v;
    x[1]=1;
    printf("\n\n Enter how many nodes (0<n<20) :");
    scanf("%d",&n);
    printf("\n\n Enter your edges(ex- u sp v) (press 'e' for end) : \n");
    for(i=1;i<=(n*n)/2;i++)
    {
        if(getchar()=='e')
            break;
        scanf("%d%d",&u,&v);
        a[u][v]=a[v][u]=1;
    }
    hamilt(2);
    printf("\n\n");
}
```

Finding Articulation Point

```

#include<stdio.h>
int d[11],num=1,b[11][11],l[11],at[11],s=1;
void art(int u,int v)
{
    int i,j=1,w,f=0;
    d[u]=num;
    l[u]=num;
    num++;
    for(i=1;b[u][i]!=0;i++)
    {
        w=b[u][i];
        if(d[w]==0)
        {
            art(w,u);
            l[u]=(l[u]<l[w])?l[u]:l[w];
        }
        else if(w!=v)
            l[u]=(l[u]<d[w])?l[u]:d[w];
        if(d[u]<=l[w])
            f=1;
    }
    if(f)
        at[s++]=u;
}
void main()
{
    int i,j,a[11][11],n,u,v,k,f=0;
    for(i=1;i<11;i++)
        for(j=1;j<11;j++)
            a[i][j]=0;
    printf("\n\n Enter how many nodes(0<n<11) :");
    scanf("%d",&n);
    printf("\n\n Enter your edges(ex- u sp v) (press 'e' for end) : ");
    for(i=1;i<=(n*n)/2;i++)
    {
        if(getchar()=='e')
            break;
        scanf("%d%d",&u,&v);
        a[u][v]=a[v][u]=1;
    }
    for(i=1;i<=n;i++)
    {
        k=1;
        for(j=1;j<=n;j++)
            if(a[i][j])
            {
                b[i][k]=j;
                k++;
            }
    }
}

```

```
        b[i][k]=0;
    }
    for(j=1,i=1;b[l][j]!=0;j++)
    {
        k=b[l][j];
        if(b[k][2])
            i++;
    }
    if(j==i)
        f=1;
    art(1,1);
    at[s]=-9;
    printf("\n\n Articulation points are : ");
    if(!f)
        for(i=1;at[i]!=-9;i++)
            printf("%3d",at[i]);
    if(f)
        for(i=1;at[i]!=1;i++)
            printf("%3d",at[i]);
    printf("\n\n");
}
```

APPENDIX C

STANDARD TEMPLATE LIBRARY (STL)



The programming languages for the contest will be C, C++, Pascal, and Java and the `scanf()/printf()` family of functions. The C++ string library and Standard Template Library (STL) is also available. This part of this book contains an introductory note about STL^[9].

Standard Template Library (STL)

The Standard Template Library, or *STL*, is a C++ library of container classes, algorithms, and iterators; it provides many of the basic algorithms and data structures of computer science. The STL is a *generic* library, meaning that its components are heavily parameterized: almost every component in the STL is a template. You should make sure that you understand how templates work in C++ before you use the STL.

Containers and algorithms

Like many class libraries, the STL includes *container* classes: classes whose purpose is to contain other objects. The STL includes the classes `vector`, `list`, `deque`, `set`, `multiset`, `map`, `multimap`, `hash_set`, `hash_multiset`, `hash_map`, and `hash_multimap`. Each of these classes is a template, and can be instantiated to contain any type of object. You can, for example, use a `vector<int>` in much the same way as you would use an ordinary C array, except that `vector` eliminates the chore of managing dynamic memory allocation by hand.

```
vector<int> v(3);           // Declare a vector of 3 elements.
    v[0] = 7;
    v[1] = v[0] + 3;
    v[2] = v[0] + v[1];     // v[0] == 7, v[1] == 10, v[2] == 17
```

The STL also includes a large collection of *algorithms* that manipulate the data stored in containers. You can reverse the order of elements in a vector, for example, by using the `reverse` algorithm.

```
reverse(v.begin(), v.end()); // v[0] == 17, v[1] == 10, v[2] == 7
```

There are two important points to notice about this call to `reverse`. First, it is a global function, not a member function. Second, it takes two arguments rather than one: it operates on a *range* of elements, rather than on a container. In this particular case the range happens to be the entire container `v`.

The reason for both of these facts is the same: `reverse`, like other STL algorithms, is decoupled from the STL container classes.

```
double A[6] = { 1.2, 1.3, 1.4, 1.5, 1.6, 1.7 };
    reverse(A, A + 6);
    for (int i = 0; i < 6; ++i)
        cout << "A[" << i << "] = " << A[i];
```

This example uses a *range*, just like the example of reversing a vector: the first argument to `reverse` is a pointer to the beginning of the range, and the second argument points one

element past the end of the range. This range is denoted $[A, A + 6)$; the asymmetrical notation is a reminder that the two endpoints are different, that the first is the beginning of the range and the second is *one past* the end of the range.

Website for downloading STL

<http://www.sgi.com/tech/stl/download.htm>

Individual files in STL

algo.h	hash_map.h	numeric	stdexcept	stl_heap.h	stl_slist.h
algorith	hash_set	pair.h	stl_algo.h	stl_iterator.h	stl_stack.h
algorithm	hash_set.h	pthread_alloc	stl_algorith	stl_iterator_base.h	stl_string_fwd.h
alloc.h	hashtable.h	pthread_alloc.h	stl_alloc.h	stl_list.h	stl_tempbuf.h
bitset	heap.h	queue	stl_bvector.h	stl_map.h	stl_threads.h
bvector.h	iterator	rope	stl_config.h	stl_multimap.h	stl_tree.h
char_traits.h	iterator.h	rope.h	stl_construct.h	stl_multiset.h	stl_uninitialized.h
concept_checks.h	limits	ropeimpl.h	stl_traits_fns.h	stl_numeric.h	stl_vector.h
container_concepts.h	list	sequence_concepts.h	stl_deque.h	stl_pair.h	string
defalloc.h	list.h	set	stl_exception.h	stl_queue.h	tempbuf.h
deque	map	set.h	stl_function.h	stl_range_error.h	tree.h
deque.h	map.h	slist	stl_hash_fun.h	stl_raw_storage_iter.h	type_traits.h
function.h	memory	slist.h	stl_hash_map.h	stl_relops.h	utility
functional	multimap.h	stack	stl_hash_set.h	stl_rope.h	valarray
hash_map	multiset.h	stack.h	stl_hashtable.h	stl_set.h	vector

Which compilers are supported?

The STL has been tested on these compilers: SGI 7.1 and later, or 7.0 with the -n32 or -64 flag; gcc 2.8 or egcs 1.x; Microsoft 5.0 and later. (But see below.) Boris Fomitchev distributes a port for some other compilers.

If you succeed in using the SGI STL with some other compiler, please let us know, and please tell us what modifications (if any) you had to make. We expect that most of the changes will be restricted to the `<stl_config.h>` header.

STL EXAMPLES**Search**

```
const char S1[] = "Hello, world!";
const char S2[] = "world";
const int N1 = sizeof(S1) - 1;
const int N2 = sizeof(S2) - 1;

const char* p = search(S1, S1 + N1, S2, S2 + N2);
printf("Found subsequence \"%s\" at character %d of sequence\n",
      S2, p - S1, S1);
```

Queue

```
int main() {
    queue<int> Q;
    Q.push(8);
    Q.push(7);
    Q.push(6);
    Q.push(2);

    assert(Q.size() == 4);
    assert(Q.back() == 2);

    assert(Q.front() == 8);
    Q.pop();

    assert(Q.front() == 7);
    Q.pop();

    assert(Q.front() == 6);
    Q.pop();
    assert(Q.front() == 2);
    Q.pop();
    assert(Q.empty());}
```

Doubly linked list

```
list<int> L;
L.push_back(0);
L.push_front(1);
L.insert(++L.begin(), 2);
copy(L.begin(), L.end(), ostream_iterator<int>(cout, " "));
// The values that are printed are 1 2 0
```

Sort

```
int A[] = {1, 4, 2, 8, 5, 7};
const int N = sizeof(A) / sizeof(int);
sort(A, A + N);
copy(A, A + N, ostream_iterator<int>(cout, " "));
// The output is " 1 2 4 5 7 8".
```

Complexity

$O(N \log(N))$ comparisons (both average and worst-case), where N is last - first.

Binary Search

```
int main()
{
    int A[] = { 1, 2, 3, 3, 3, 5, 8 };
    const int N = sizeof(A) / sizeof(int);

    for (int i = 1; i <= 10; ++i) {
        cout << "Searching for " << i << ": "
              << (binary_search(A, A + N, i) ? "present" : "not present") <<
endl;
    }
}
```

The output

```
Searching for 1: present
Searching for 2: present
Searching for 3: present
Searching for 4: not present
Searching for 5: present
Searching for 6: not present
Searching for 7: not present
Searching for 8: present
Searching for 9: not present
Searching for 10: not present
```

APPENDIX D

PC² CONTEST ADMINISTRATION AND TEAM GUIDE



PC² is a dynamic, distributed real-time system designed to manage and control Programming Contests. It includes support for multi-site contests, heterogeneous platform operations including mixed Windows and Unix in a single contest, and dynamic real-time updates of contest status and standings to all sites. Here we describe the steps required to install, configure, and run a contest using PC². Further information on PC², including how to obtain a copy of the system, can be found at <http://www.ecs.csus.edu/pc2>.

Programming Contest Judge Software - PC²

PC² operates using a client-server architecture. Each site in a contest runs a single PC² *server*, and also runs multiple PC² *clients* which communicate with the site server. Logging into a client using one of several different types of PC² accounts (Administrator, Team, Judge, or Scoreboard) enables that client to perform common contest operations associated with the account type, such as contest configuration and control (Administrator), submitting contestant programs (Team), judging submissions (Judge), and maintaining the current contest standings (Scoreboard).

PC² clients communicate only with the server at their site, regardless of the number of sites in the contest. In a multi-site contest, site servers communicate not only with their own clients but also with other site servers, in order to keep track of global contest state. The following communication requirements must therefore be met in order to run a contest using PC²: (1) a machine running a PC² server must be able to communicate via TCP/IP with every machine running a PC² client at its site; and (2) in a multi-site contest, every machine running a PC² server must be able to communicate via TCP/IP with the machines running PC² servers at every other site^[1]. In particular, there must not be any firewalls which prohibit these communication paths; the system will not operate if this communication is blocked. It is not necessary for client machines to be able to contact machines at other sites.

Each PC² module (server or client) reads one or more .ini initialization files when it starts; these files are used to configure the module at startup. The client module also tailors its configuration when a user (Team, Judge, etc.) logs in. In a typical PC² contest configuration, each Team, Judge, etc. uses a separate physical machine, and each of these machines runs exactly one client module. It is possible to have multiple clients running on the same physical machine, for example by having different users logging in to different accounts on a shared machine. In this case, each user (Team, Judge, etc.) will be executing their own Java Virtual Machine (JVM), and must have their own separate directory structure including their own separate copy of the PC² initialization files in their account.

PC² Setup for Administrator

For those people who hate to read manuals and would rather take a chance with a shortcut list, here is a *very terse* summary of the steps necessary to install PC² and get it running. Please note that this is provided as a convenience for geniuses (or gluttons for punishment). The remainder of the manual was written to help everyone else. If you have

* For further information about PC² please check : <http://www.ecs.csus.edu/pc2>

problems after following this list, *please read the rest of the manual* before sending us a request for help.

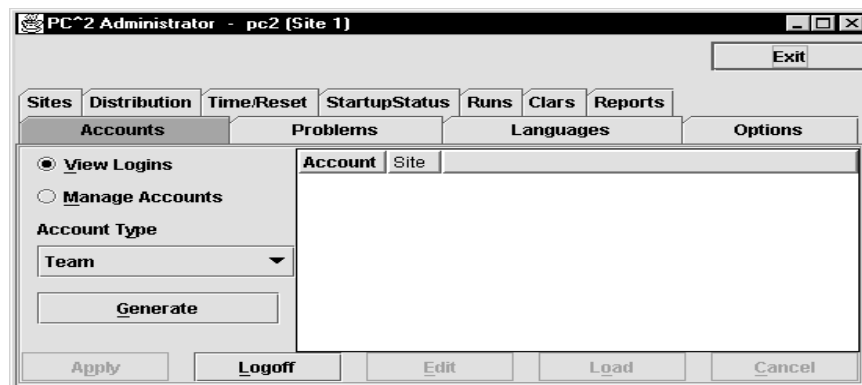
- ❑ Install Java (version 1.3.1 or greater) ;
- ❑ Install PC² by unzipping the PC² distribution to the PC² installation directory;
- ❑ Add the Java *bin* directory and the PC² installation directory to the PATH;
- ❑ Add java/lib, and the PC² installation directory to the CLASSPATH;
- ❑ Modify the *sitelist.ini* file as necessary to specify each site server name.
- ❑ Edit the *pc2v8.ini* file to point servers and clients to the server IP:port and to specify the appropriate site server name; put the modified .ini file on every server and client machine;

```
# sample pc2v8.ini file for site named Site

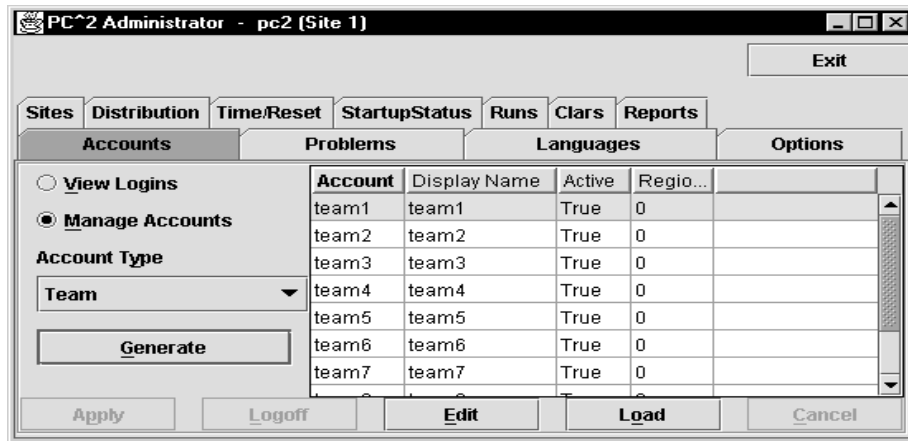
[client]
# tell the client what site it belongs to
# and where to find its server (IP and port)
site=Site1
server=192.168.1.117:50002

[server]
# tell the server which site it is serving
site=Site1
```

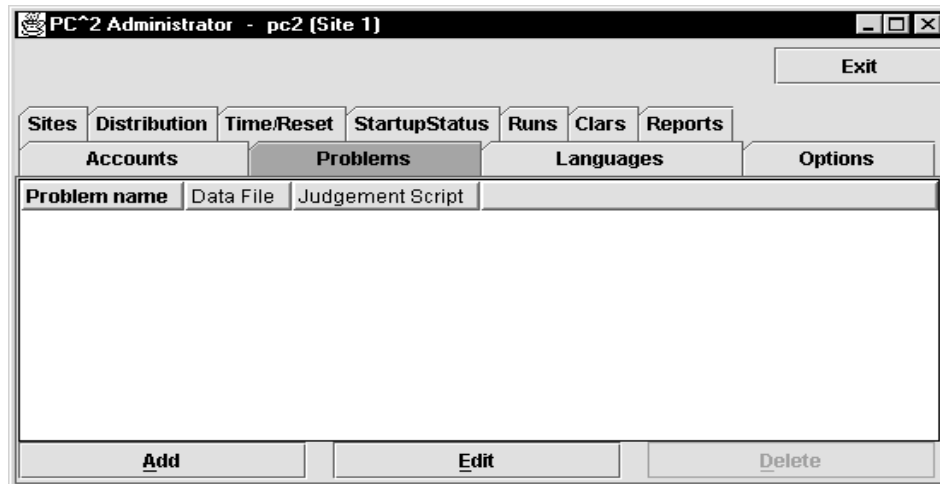
- ❑ Start a PC² server using the command pc2server and answer the prompted question.
- ❑ Start a PC² Admin client using the command pc2admin and login using the name root and password root.



- ❑ Configure at least the following contest items via the Admin:
- ❑ Accounts (generate the necessary accounts);

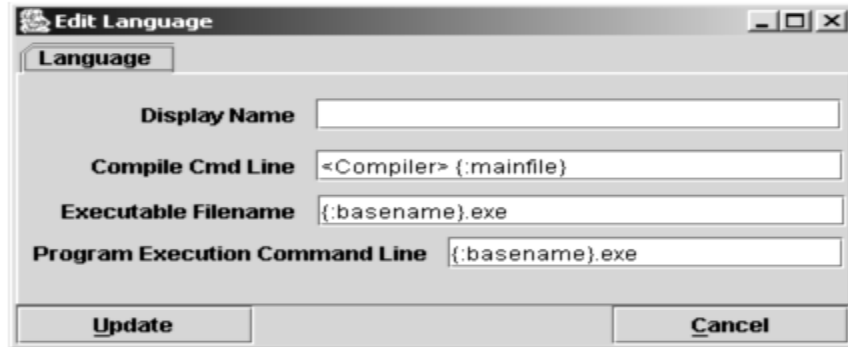


- ❑ Problems (create one or more contest problems, specifying the problem input data file if there is one);



- ❑ Languages (create contest languages, specifying the language name, compile command line, executable filename, and execution command line).

❑ Language Parameter for Turbo C/C++ : `tcc -Ic:\tc3\bin -Lc:\tc3\lib {;mainfile}`

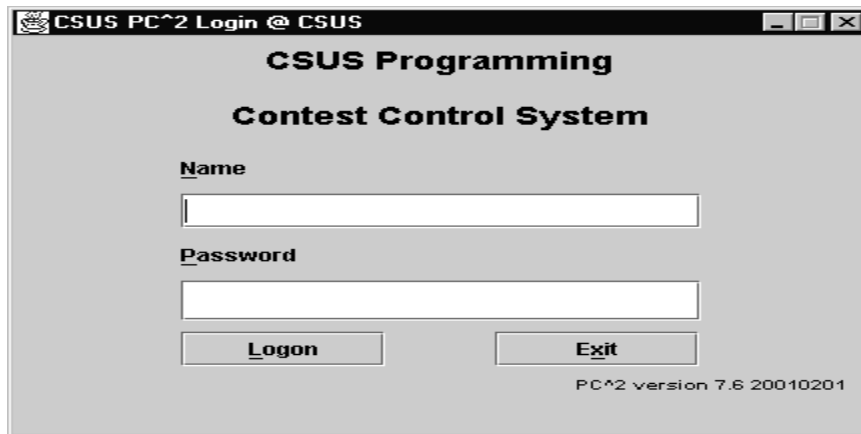


- Press the Start Contest button on the Admin Time/Reset tab;
- Start a PC² client on each Team and Judge machine and log in using the Admin-created accounts and passwords.
- Start a PC² client on the Scoreboard machine and log in using the board1 Scoreboard account/password; arrange for the scoreboard-generated HTML files to be accessible to users browsers.



PC² Team Guide (For Contestants)

This guide is intended to familiarize you with the process of submitting programs to Contest Judges using the PC² (“P-C-Squared”) **P**rogramming **C**ontest **C**ontrol system. Starting PC² will bring up the PC² **login screen**, shown below:



CSUS PC² Login @ CSUS

**CSUS Programming
Contest Control System**

Name

Password

Logon **Exit**

PC² version 7.6 20010201

To login to PC², click once on the **Name** box on the login screen, enter your assigned team ID, press the TAB key or click on the **Password** box, then enter your assigned password. Your team ID will be of the form **teamxx**, where xx is your assigned team number (for example, “**team3**” or “**team12**”). After entering your team name and password, click on the **Logon** button.

Submitting a Program to the Judges

Once the system accepts your login, you will be at the PC² **Main Menu** screen, shown below. Note that the team ID (“team1” in this case) and the team’s site location (“CSUS” in this case) are displayed in the title bar.



PC² - team1@CSUS

Exit

Submit **Clarifications** **Runs** **Settings**

Problem **Bowling**

Language **Java**

Main File **Select** C:\work\bowling.java

Additional Files

Add **Remove**

Test **Submit**

Clicking on the **SUBMIT** tab near the top of the screen displays the **Submit Run** screen, which is shown above.

Clicking in the **Problem** field will display a list of the contest problems; choose the problem for which you wish to submit a program (“run”) to the Judges (in the example, a problem named “Bowling” has been chosen).

Clicking in the **Language** field will display a list of the programming languages allowed in the contest; choose the language used by the program that you wish to submit to the Judges (in the example, “Java” has been chosen).

To submit a program to the Judges, you must specify the name of the file containing your **main program**. Click on the **Select** button to invoke the “File Dialog” which lets you locate and select your main file. The Dialog lets you automatically navigate to the correct path and file location (in the example, the main program file “C:\work\bowling.java” has been selected).

If your program consists of more than one file, you must enter the additional file names in the **Additional Files** box. Click the **Add** button to invoke the dialog which lets you locate and select your additional files; select a file in the box and click the **Remove** button to remove a previously-selected file.

Important: you *must* specify the name of your *main program* file in the **Main File** field, ***not*** in the **Additional Files** box! Select only source code files for submission to the Judges. Do not submit data files or executable files.

PC² Uninstall

To uninstall PC² it is only necessary to undo the above installation steps; that is, remove the \$PC2HOME directory and its contents and restore the system environment variables to their former values. PC² itself does not make any changes to any machine locations outside those listed above either during installation or execution. In particular, for example, it makes no entries in the registry in a Windows environment, nor does it copy any files to locations outside the installation directories in any environment.

APPENDIX E

IMPORTANT WEBSITES/ FOR ACM/ICPC PROGRAMMERS



Now a days the world is open for learners and the Internet makes the world inside our room. There are many websites now on the Internet for the beginners as well as programmers. Some sites are for contest and tutorial, some for books, some sites are for programming language. Mathematics is one of the essential for programming and there are a lot of mathematical site in the net. The pioneers also publish their ideas and advice on the Internet for us. In this page you will find the link of those pages.^[10]

Online Judges And Tutorial Sites

<http://online-judge.uva.es/> and <http://cii-judge.baylor.edu/>

University of Valladolid 24-hour Online Judge and Official ACM/ ICPC Live archive contains all problems of ACM Regionals.

<http://acm.timus.ru/>

24-hour Online Judge hosted by Ural State University of Russia. This site contains the problems from many Russian contests. This can be a very useful site for students to practice. From this site we can understand how these Russian Problems differ from non-Russian problems.

<http://plg.uwaterloo.ca/~acm00/>

This is the contest page of University of Waterloo. This university currently leads the contest arena of the World. They arrange programming contests almost in every alternate term and problems of these contests set standard for many other universities.

<http://oldweb.uwp.edu/academic/mathematics/usaco/> and <http://ace.delos.com/>

Online training site for contests which guides beginners with a step by step problem solution.

<http://www.acmsolver.org/>

Another training site for 24-hour online programming contests developed by "Ahmed Shamsul Arefin" mainly for novice programmers now linked with world's major online judges.

<http://www.acm.inf.ethz.ch/ProblemSetArchive.html>

This site is the best resource for the past ACM Regionals and World Finals problem set. It has also solutions and judge data for many contests, which will help us for practice.

<http://acm.uva.es/problemset/replies.html>

This page contains the information about the meaning of different judge replies. This is also a link page of the site acm.uva.es.

<http://contest.uvarov.ru/>

This is the POTM master's home page. POTM Contest means Programmer of the Month Contest. The problem set of this contest is different then the conventional programming contest. The organizer gives more stress on Optimization than number of solution.

<http://www.acmbeginner.tk>

Is it very difficult to disclose oneself as a good programmer? To solve the problems, the difficulty that you face in the 24-hour online judge and the help and guideline that found to overcome them is integrated in this site "ACMBEGINNER" by M H Rasel.

<http://groups.yahoo.com/group/icpc-l>
http://groups.yahoo.com/group/acm_solver

You talk about algorithms, data structures and mathematical concepts. The group is made for teams, ex-teams and everybody interested in on-line programming. Coaches and teachers are very welcome.

Site of Pioneers !

Newcomers advice, by Shahriar Manzoor

One of the greatest programmer/ problemsetter of acm.uva.es as well as a renowned progeammer of Bangladesh, advices the newcomers. He is now the judge of UVa OJ site and WF. He is also the elite panel problem setter of the site acm.uva.es. you can visit his publication by the using the URL <http://www.acm.org/crossroads/xrds7-5/>

World of Steven, by Steven Halim

Steven Halim published his site for the problem solvers including the problemset tricks tutorials with the difficulty level. In his site he has also categorized problems. This site contains almost all hints of the problems that he has solved. You can visit the site using the link <http://www.comp.nus.edu.sg/~stevenha/programming/acmoj.html>

Reuber Guerra Duarte

Reuber Guerra Duarte developed this site with some solutions (the real source code) for some problems that already solved by him. Please don't directly send his solutions to judge. <http://www.dcc.ufmg.br/~reuber/solutions/index.html>

Ed's Programming Contest Problem Archive by Ed-Karrels

Ed-karrel one of the greatest problem setter publish this site with the solutions to many problems. visit the site with the URL http://www.karrels.org/Ed/ACM/prob_index.html

Mark Dettinger

A living legend of programming contest. He was a main contest architect of the University of ULM, Germany. Now this responsibility lies on another person Water Guttman. If you visit the site you will find many useful link as well as many interesting incidents of his life and onmces again you will realize again that the boy gets eleven in math out of hundred can also be genius. URL <http://www.informatik.uni-ulm.de/pm/mitarbeiter/mark/>

Books**Introduction to Algorithm (CLR)**

Introduction to Algorithms (MIT Electrical Engineering and Computer Science Series) by Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest. Now they already publish the 2nd Edition + Stein as the 4th author. This book is a-must-have to be a good programmer.

Algorithm Design Manual

The Algorithm Design Manual by Steven S. Skiena (+ book website). Also a-must-have for every good programmer.

The Art of Computer Programming

The Art of Computer Programming, Volumes 1-3 Boxed Set by Donald E. Knuth. Hard to understand for beginner but worth to read

Concrete Mathematics

Concrete Mathematics, Second Edition + by Ronald L. Graham, Donald E. Knuth, Oren Patashnik. This book contain mathematical problem and solutions, but This is very hard to understand. Though it is hard, it is very useful for programming contest. Several formulation techniques discussed here.

Number Theory

Number Theory by S G Telang is a book, consists of lot of knowledge about the number theory. It is better idea to read this book before reading the book Concrete Mathematics. This is easy to understand.

Discrete Mathematics And Its Application

Discrete Mathematics And Its Application by Kenneth H. Rosen consists of a lot of practical help on integer matrices, mathematical reasoning, counting, graph tree problems.

Data Structure Using C and C++

Data structure using C and C++ by Yedidyah Langsam, Moshe J. Augenstein Aaron M. Tenenbaum. Second Edition. This book contains basic problem related to Data Structures with C coding.

Programming Challenges

Programming Challenges, by Steven Skiena Miguel Revilla. We dont want to say anything about this two legend. All the programmers are familiar with this two ultimate Programming Contest Legends. Use this book to learn what to do for the programming contest.

Art of Programming Contest

Well, Art of Programming Contest is this one, by Ahmed Shamsul Arefin.

Programming Language Guides

<http://www.sgi.com/tech/stl/>

In this site you will find the STL documentation. The STL is very important for programming. It is efficient and make code simple and sort. To use this the C++ template knowledge is necessary. The beginner can overlook the STL at beginning but is is needed in future.

www.cplusplus.com / www.cprogramming.com

Great sites for the C/C++ programmers. These sites contains documents, References, sources codes with electronic forum where you can consult with great programmers.

Mathematics Site

mathworld.wolfram.com

MathWorld is a comprehensive and interactive mathematics encyclopedia intended for students, educators, math enthusiasts, and researchers. Like the vibrant and constantly evolving discipline of mathematics, this site is continuously updated to include new material and incorporate new discoveries.

<http://mathforum.org/>

The Math Forum is a leading center for mathematics and mathematics education on the Internet. Its mission is to provide resources, materials, activities, person-to-person interactions, and educational products and services that enrich and support teaching and learning in an increasingly technological world.

<http://www.math.com/>

Math.com is dedicated to providing revolutionary ways for students, parents, teachers, and everyone to learn math. Combining educationally sound principles with proprietary technology, Math.com offers a unique experience that quickly guides the user to the solutions they need and the products they want.

<http://www.math-net.org/>

Math-Net intends to coordinate the electronic information and communication activities of the global mathematical community with the aim to enhance the free-flow of information within the community.

REFERENCES

- [1] “HOW TO DO BETTER IN 24 HOURS ONLINE JUDGES”, Anupam Bhattacharjee, , Published by ACMSolver.org, 2003.
- [2] “World of Seven”, METHODS TO SOLVE VALLADOLID ONLINE JUDGE PROBLEMS, Steven Halim, National University of Singapore, <http://www.comp.nus.edu.sg/~stevenha/>
- [3] ACMSolver.org, ACM/ICPC Programming Contest Tutorial Website for Valladolid OJ by Ahmed Shamsul Arefin <http://www.acmsolver.org>
- [4] “Theoretical Computer Science Cheat Sheet”, Steve Seiden, <http://www.csc.lsu.edu/~seiden>
- [5] “Teach Yourself C in 21 Days”, Peter Aitken, Bradley L. Jones, website: <http://www.mcp.com/info/0-672/0-672-31069-4/>
- [6] “Common Mistakes in Online and Real-time Contests”, Shahriar Manzoor, ACMCross Roads Student Magazine, www.acm.org/crossroads/xrds7-5/contests.html
- [7] Verhoeff, T. Guidelines for Producing a Programming-Contest Problem Set: <http://www.wpa.win.tue.nl/wstomv/publications/guidelines.html>
- [8] Ernst, F., J. Moelands, and S. Pieterse. Teamwork in Programming Contests: 3 * 1 = 4, Crossroads, 3.2. <http://www.acm.org/crossroads/xrds3-2/progcon.html>
- [9] STL (Standard Template Library), Silicon Graphics, Inc, <http://www.sgi.com/tech/stl/>
- [10] ACMBeginner.tk, ACM Valladolid Online Judge (OJ) Tools, Tips and Tutorial by M H Rasel, <http://www.acmbeginner.tk/>
- [11] Brian W. Kernighan- “Programming in C: A Tutorial” <http://www.lysator.liu.se/c/bwk-tutor.html>
- [12] Rob Kolstad , USACO Training Gateway, <http://ace.delos.com/>

INDEX

- ADHOC problems, 16
- Adjacency List, 139
- Adjacency Matrix, 139
- Algorithm, 19
- Array, 74
- Base Number Conversion, 91
- Bellman-Ford Algorithm, 164
- Big Integer, 92
- Big Mod, 91, 195
- Binary Search, 113
- Binary Search Tree, 113
- Breadth-first search, 144
- BRUTE FORCE METHOD, 85
- Bubble Sort, 107
- Carmichael Number, 93
- Collision Handling, 115
- Combinations, 202
- COMPILATION ERROR (CE), 15
- Connected Fields, 158
- Connectedness, 141
- Convex Hull, 173
- Counting Change, 131
- Counting Combinations, 94
- Counting Sort, 111
- DATA STRUCTURES, 72
- debugging, 15
- Decimal To Roman, 198
- Decomposition, 89
- Depth First with Iterative Deepening, 150
- Depth-first search, 147
- Dictionary, 114
- Dijkstra Algorithm, 163
- Directed Acyclic Graph, 165
- Directed Graph, 137
- Divide and Conquer, 88
- Divisors and Divisibility, 95
- DYNAMIC PROGRAMMING, 121
- Earth coordinate system, 172
- Edge List, 138
- Edit Distance, 127
- Euclid's Algorithm, 190
- Euler Cycle, 170
- Euler Path, 170
- Exponentiation, 96
- Factorial, 97
- Fermat Algorithm, 104
- Fermat Little Test, 104
- Fibonacci, 97
- Floyd Warshall, 165
- GCD, 191
- Graph Transpose, 170
- GRAPHS, 134
- Greatest Common Divisor (GCD), 99
- GREEDY ALGORITHMS, 117
- Hash Table variations, 116
- Infix to Postfix conversion, 100
- Informed Search, 154
- Integer to any base, 197
- Josephus Problem, 200
- Judges, 26
- Kruskal's algorithm, 161
- LCM, 191
- LCS, 205
- Linear-time Sorting, 111
- Linked List, 76
- Longest Common Subsequence, 125
- Longest Inc/Dec-reasing Subsequence (LIS/LDS), 128
- Lowest Common Multiple (LCM), 99
- Matrix Chain Multiplication, 123
- Matrix Chain Multiplication Problem, 123
- Maximum Interval Sum, 132
- MCM, 209
- MiniMax Distance, 167
- Minimum spanning trees, 160
- Minimum Spanning Trees, 159
- Multiple input programs, 83
- Number Conversions, 196
- optimal sub-structure, 124
- Optimizing, 89
- order of growth, 22
- Other Dynamic Programming Algorithms, 133
- PC², 235
- Postfix Calculator, 100
- Pre-Computation, 89
- Prime Factors, 101
- Prime Numbers, 102
- prime testing, 102
- Prim's algorithm, 162
- Queue, 79
- Quick Sort, 108
- Radix Sort, 111
- Recursion, 86
- SEARCHING, 113
- Sieve of Eratosthenes, 105
- SORTING, 106
- sorting algorithms, 106
- Stack, 78
- STL, 231
- Strongly Connected Components, 171
- Subgraphs, 142
- Symmetries, 90
- Topological Sort, 119, 171
- Transitive Hull, 166
- Uninformed Search, 143
- Valladolid OJ, 16
- Variations on Binary Trees, 114
- Zero-One Knapsack, 130